Keyloop Technical Assessment

Welcome to the Keyloop technical assessment! This challenge is designed to simulate the real-world problems our engineers solve every day. It's an opportunity for you to showcase your design thinking, technical execution, and problem-solving skills on a task that is relevant to our mission of transforming the automotive retail experience.

A Modern Approach: Engineering with AI Agents

Generative AI (GenAI) tools are now integral to the modern software development workflow. This challenge requires you to use GenAI tools as an essential collaborator. You might use them to generate boilerplate code, refactor, or even have an AI agent implement the entire solution based on your direction.

We will evaluate not only your core technical skills but, more importantly, your ability to strategically direct, validate, and take ownership of an AI-generated solution. Your process for guiding and verifying the AI's work is a primary evaluation criterion.

A Note on Ambiguity: These scenarios are designed to mimic real-world requirements, which can be ambiguous. If a requirement is unclear, please make a reasonable assumption and document it in your System Design Document.

Suggested Free AI Coding Tools: Not sure where to start? Here are some free tools you can use. Feel free to use any tool you prefer.

- OpenCode: The open source AI coding agent
- Google Antigravity: Agentic AI-powered IDE by Google
- Gemini CLI: Build, debug and deploy with AI in the terminal
- GitHub Copilot: Your AI pair programmer
- Kiro: Agentic AI development from prototype to production

Choose Your Challenge Scenario

Please select **one** of the following four scenarios to design and build.

Scenario A: The Unified Service Scheduler

- **Domain:** Ownership
- **Task:** Build an Appointment Scheduler application to replace manual booking systems.
- **Core Requirements:**
    1. **Resource Constrained Booking:** Allow a user to request a service appointment for a specific vehicle, service type, and dealership at a desired time.

2. **Real-Time Availability Check:** Before confirming, check for the availability of both a ServiceBay and a qualified Technician for the entire service duration.
3. **Confirmed Appointment Record:** Upon success, create a persistent Appointment record associating the customer, vehicle, technician, and service bay.

Scenario B: The Intelligent Inventory Dashboard

- **Domain:** Supply
- **Task:** Build an Intelligent Inventory Dashboard to give dealership managers a real-time overview of their vehicle stock.
- **Core Requirements:**
    1. **Inventory Visualization:** Display a filterable list of all vehicles in a dealership's inventory (e.g., filter by make, model, age).
    2. **Aging Stock Identification:** Automatically identify and prominently display "aging stock" (vehicles in inventory for >90 days).
    3. **Actionable Insights:** Allow a manager to log and persist a status or proposed action for each aging vehicle (e.g., "Price Reduction Planned").

Scenario C: The Sales Lead Management Tool

- **Domain:** Demand
- **Task:** Build a lightweight Sales Lead Management Tool to help salespeople manage and track incoming leads from the dealership's website.
- **Core Requirements:**
    1. **Lead Inbox:** Display a list of all incoming sales leads.
    2. **Lead Details View:** Clicking a lead must show its full details and a chronological log of all follow-up activities.
    3. **Activity Logging:** Provide an interface for a salesperson to log a new follow-up activity for a lead (e.g., "Called customer"), which must be persisted.

Scenario D: The Unified Document Viewer

- **Domain:** Operate
- **Task:** Build a Unified Document Viewer to provide a single view of all documents related to a vehicle by connecting to two different (mocked) dealership systems.
- **Core Requirements:**

1. **Unified Search:** Provide a single search interface where a user can enter a Vehicle Identification Number (VIN).
2. **Data Aggregation:** The backend must make parallel requests to two mocked external APIs (a "Sales System API" and a "Service System API").
3. **Aggregated View:** The UI must display a single, consolidated list of all documents from both sources, clearly indicating the source system for each.

The Challenge Structure

Your work should be based on the single scenario you chose.

Part 1: System Design

Produce a **System Design Document** that outlines your architectural plan. You are free to choose the format (e.g., Markdown, PDF, diagram). This document should include:

- An architecture diagram.
- A brief description of each component's role.
- An explanation of the data flow.
- A list of your chosen technologies with justifications.
- Your strategy for observability (e.g., logging, metrics, tracing).
- A dedicated section describing how you used GenAI to assist in the design phase.

Part 2: Service Implementation (Your Choice)

Choose **one service layer** to implement fully: either the **backend** or the **frontend** and mock or stub the other. Your implementation should fulfil the acceptance criteria of your chosen scenario for the service layer you select.

- **If you choose Backend:** Expose a RESTful API and use a persistent database. Mock or stub the client-side layer with a simple test harness, cURL examples, or a basic API contract (e.g., OpenAPI spec).
- **If you choose Frontend:** Build a web application that demonstrates the full user experience for your scenario. Mock the backend layer using static data, a mock API library, or a local JSON server.
- **Build For the Future:** Whichever layer you choose, your design and implementation should consider scalability, performance, reliability, maintainability, and observability.

Deliverables & Submission

Please submit the following three artifacts:

1. System Design Document: Your architectural plan.
2. Working Code: A Git repository containing your chosen service implementation. It must include:
    - A README.md with clear instructions on how to build, run, and test your application.
    - A dedicated section in the README for your AI Collaboration Narrative. Describe your high-level strategy for guiding the AI, your process for verifying and refining its output, and how you ensured the final quality of the code.
    - A suite of tests that validate the core business logic.
3. Video Submission (5-10 minutes): A short video presentation covering:
    - A brief introduction to yourself and your chosen scenario.
    - A walkthrough of your system design and implementation highlights.
    - A summary of your AI collaboration story (1-2 minutes).
    - A brief demonstration of your application.
    - What you learned and any challenges you faced.

Evaluation Framework

We will be evaluating your submission across four key dimensions:

1. Problem Solving & System Design: The clarity, logic, and foresight of your architecture.
2. Technical Execution: The quality, correctness, and testing of your implementation.
3. AI Engineering & Verification: Your strategy for directing AI and your process for verifying, debugging, and owning the final solution.
4. Communication & Presentation: The clarity and professionalism of your documentation and video.

We are excited to see what you build. Good luck!